

Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 31/01/2020

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially?	☐ Yes	☐ No
Professor:	☐ Carminati	☐ Zanero
Course:	☐ Milan	☐ Como

Instructions

- **Duration:** 2 hours
- The exam is composed of 15 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed the homework, by putting an "X" mark appropriately.
- The exam is "closed books." Please put away in a non-suspicious place (i.e., not below the desk) any notebook, or similar. You will be expelled if, at any time, if you do not follow this rule.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Shut down and store electronic devices. They will be subject to inspection if found, and you may be expelled if you are found using one.
- Please answer within the allowed space. Schemes are good, short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- The answers should be written exclusively in the space provided below the questions.
- Always motivate your answer.
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next call.**

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (9 points)

Consider the C program below:

```
00  #include <stdio.h>
01  #include <stdlib.h>
02  #include <string.h>
03
04  void usr_in(int sel, char *msg) {
05      char usr[4];
06      if (sel) {
07          printf("Welcome message: \n");
08          printf(msg);
09          gets(usr);          /* Insert username */
10      }
11  }
12
13  void login(int sel, char *msg) {
14      struct {
15          char pass[4];
16          char buf[4];
17          char *error;
18      } s;
19      s.error = "Access Denied!";
20      strcpy(s.pass, "psw\0");
21      usr_in(sel, msg);
22      scanf("%7s", s.buf);    /* Insert password*/
23      if(!strcmp(s.buf, s.pass)){
24          printf("Access Granted!");
25          return;            /* Access OK */
26      }
27      else {
28          printf("%s", error);
29          abort();           /* Access KO */
30      }
31  }
32
33  int main(int argc, char** argv) {
34      if (strlen(argv[2]) < 8){
35          login(atoi(argv[1]), argv[2]);
36      }
37      abort();
38  }
```

Assume that:

- The C standard library is loaded at a known address during every execution of the program, and that the (exact) address of the function `system()` is `0xfee48da0`.
- Environment variables are located in the highest addresses.

1. [2 points] The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table:

Vulnerability	Line	Motivation
Buffer Overflow		
Format String		

2. [1 point] Assume that the program is compiled for (and executed in) the usual IA-32 architecture (32-bits), with the usual cdecl calling convention. Draw the stack layout just before the execution of line 06, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable (i.e., EBP, ESP);
- The content of relevant registers;
- The content of the functions stack frames (`main`, `login`, `user_in`), and their boundaries (just focus on what matters for the exploitation of typical memory corruption vulnerabilities).

--

3. [2 points] Assume that the program is compiled and run with no mitigation against exploitation of memory corruption vulnerabilities (**no canary, executable stack, environment with no ASLR active**). Write an exploit **for the buffer overflow vulnerability you identified** in order to bypass the authentication mechanism. Your exploit **must** execute the following simple shellcode, composed only by 12 instructions (12 bytes) that launch the program `/bin/secret`:

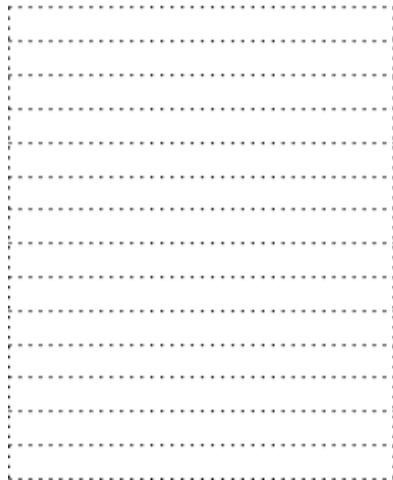
0x31 0xc0 0x6a 0x03 0x5b 0xb0 0x6e 0xcd 0x80 0xfa 0xeb 0xfe

Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.

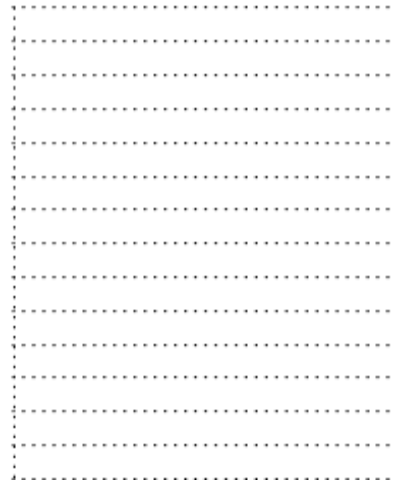
Command line argument (argv[1]):

Command line argument (argv[2]):

Stack layout before vulnerable line



Stack layout after vulnerable line



5 [1 point] Now assume that the stack is not executable. Is the exploit you just wrote still working without modifications? If so, explain why. If not, explain why and propose an alternative solution to launch the program `/bin/sh`. Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.

--

--

Exercise 2 (9 points)

BookMeUp is a web service that allows students, after having created a personal account, to book office hours with their course instructors. In line with the most recent web development trends, BookMeUp is developed as a thin HTTP Application Programming Interface (API) with a “static” frontend written in HTML+Javascript; as the API is meant to be used programmatically from a JavaScript frontend, all API requests (be it GET or POST ones) return JSON-formatted responses¹ rather than HTML documents. The API is composed of various endpoints (i.e., URLs). You can find below a short description and the implementation (in a Python-like pseudocode) of the endpoints relevant for this exercise.

`https://bookmeup.example.com/api/register` (supported method: POST)

Registers a new user (student). Parameters: name, password.

```
01 def register(request, response):
02     response.set_header('Content-Type: application/json')
03     password = sha256sum(request.post['password'] + config.PASSWD_HASH_SALT)
04     try:
05         db.query("insert into users(`name`, `password`) " + \
06                 "values ('" + request.post['name'] + "', '" + password + "');"
07     except:
08         return json.dumps({'result' : 'fail'})
09     return json.dumps({'result' : 'ok'})
```

`https://bookmeup.example.com/api/login` (supported method: POST)

Log in a user. Parameters: name, password. Sets cookies with the session information (user ID / token).

```
10 def login(request, response):
11     response.set_header('Content-Type: application/json')
12     password = sha256sum(request.post['password'] + config.PASSWD_HASH_SALT)
13     res = db.query("select id, role from users where name = '" + \
14                   + request.post['name'] + "' and password = '" + password + "' limit 1;")
15     if res.get_num_rows() != 1:
16         return json.dumps({'status' : 'fail'})
17     uid, role = res.get_first_row()
18     response.set_cookie('user_id', uid)
19     response.set_cookie('signature', sha256sum(uid + config.SECRET_KEY))
20     return json.dumps({'status' : 'ok', 'id' : uid, 'role' : role})
```

`https://bookmeup.example.com/api/slots` (supported method: GET)

¹ JSON (JavaScript Object Notation) is a format to represent in plain text arbitrary dictionary-like objects composed of multiple key-value pairs; values can be simple data such as numbers or strings, other JSON objects, arrays of JSON objects. Think of them as a way to encode (serialize) something similar to a C-language struct as text. A simple JSON object is: `{"type" : "professor", "name" : "Stefano", "surname" : "Zanero"}`.

Returns a list of the available slots for a given instructor (given its user ID as a GET parameter)

```
21 def slots(request, response):
22     response.set_header('Content-Type: application/json')
23     instructor_id = request.get['instructor_id']
24     res = db.query("select id, timestamp from slots " + \
25                   "where user = " + instructor_id + " and available = TRUE")
26     slots = []
27     for slot_id, timestamp in res:
28         slots.append({'id' : slot_id, 'time' : timestamp })
29     return json.dumps({'instructor' : instructor_id, 'slots' : slots})
```

<https://bookmeup.example.com/api/book>

(supported method: POST)

Books an appointment with an instructor. Parameters: instructor ID, appointment slot ID. User must be logged in.

```
30 def book(request, response):
31     response.set_header('Content-Type: application/json')
32     student_id = request.cookies['user_id']
33     if sha256sum(student_id + config.SECRET_KEY) != request.cookies['signature']:
34         return json.dumps({'result' : 'fail'})
35     slot = request.post['slot']
36     instructor = request.post['instructor']
37     try:
38         db.query("insert into appointments (`instructor`, `student`, `slot`) " + \
39               "values (" + instructor + ", " + student_id + ", " + slot + ");"
40         db.query("update slots set available = FALSE where slot = " + slot + ";")
41         return json.dumps({'result' : 'ok'})
42     except:
43         return json.dumps({'result' : 'fail'})
```

Assumptions:

- The variables `request.get` and `request.post` store the parameters passed through a GET or POST request; `request.cookie` contains the cookies sent with the request; `response.set_header` and `response.set_cookie` allow to add arbitrary headers and cookies to the response.
- The function `json.dumps` converts a Python-like dictionary into a string formatted as JSON

The application uses a relational database for storage, accessible through the function `db.query` via standard SQL queries. Below the database schema:

USERS(id, name, password, role) *primary key: id (integer autoincrement) - default value for role: "student"*

SLOTS(id, user_id, timestamp, available) *primary key: id (integer autoincrement)*

APPOINTMENTS(slot_id, student_id, instructor) *primary key: slot_id and student_id (composite key)*

Please read the following questions and then answer one by one.

1. [3 points] Only considering code snippet of all the pages, identify which of the following web application vulnerabilities are present:

<i>Vulnerability class</i>	<i>Is there a vulnerability belonging to this class in the code? If so, explain why it is present and specify how an adversary could exploit it. Specify also the vulnerable lines involved.</i>	<i>If the vulnerability is present in the code above, explain the simplest procedure to remove this vulnerability. Otherwise write “No vulnerability”</i>
Cross-site scripting (XSS) - please specify whether stored or reflected	Lines:	
Cross site Request Forgery (CSRF)	Lines:	
SQL Injection	Lines:	

2. [1 point]. Exploiting one of the vulnerabilities detected before, write down an exploit that allows an **unauthenticated user** to *log in* as the user “**s.zanero**”.

3. [1 point]. Exploiting one of the vulnerabilities detected before, write down an exploit that allows an **arbitrary user** to **view** the user IDs of **all the users that have, or had, an appointment with “m.carminati”** (without logging in as m.carminati!)

4. [1 point]. Assume you are the database administrator, and have no way to modify the source code of the application. Can you mitigate the vulnerabilities you exploited in points 2. and 3.? Which ones? How?

5. [1 point]. Thanks to a marvelous application security team, you fixed all the vulnerabilities found in point 1 right after the discovery. Unfortunately, the BookMeUp security response team receives a message from an instructor, complaining that **students are exploiting a vulnerability in the application to book *already occupied* appointment slots**. Please find this additional vulnerability, describe how it works, how it can be exploited, and what is the simplest way to fix it.

Now consider the client-side **API frontend**, written in HTML and Javascript (i.e., client-side code). For simplicity, below you can find a code snippet of the part of the frontend that allows to view the slots of a given user, whose ID is specified in the URL bar (as part of `location.hash`).

For instance, if the current URL is <https://bookmeup.example.com/slots.html#id=10>, the code snippet below will (a) fetch from the API the appointment slots for the instructor with user ID 10, (b) generate an HTML snippet with the data retrieved from the API, (c) insert it into the DOM.

```
/* location.hash is the part of the URL following the hash sign ("id=10" in the example) */
var instructor_id = location.hash.substr(location.hash.search("id=") + "id=".length)
/* fetch() performs a GET request and handles the result (parsed as JSON) */
var url = "https://bookmeup.example.com/api/slots?instructor_id = " + instructor_id;
var response = fetch_and_parse_json(url);
/* result contains the parsed JSON as a Javascript object */
var str = "<h1>Available slots for user ID:" + instructor_id + "</h1>";
str += "<table><tr><td>ID</td><td>Date/time</td><td></td></tr>";
response.slots.forEach((el) => {
    /* executed for each element of the array result.slots (ie, it's a for loop) */
    str += "<tr><td>" + el.id + "</td>" + "<td>" + el.timestamp + "</td>" +
        '<td><a href="/book.html#id=' + instructor_id +
        '&timestamp=' + el.timestamp + '">Book now!</a></td></tr>';
})
str += "</table>";
/* update the page */
document.write(str);
```

6 [2 points]. Clearly, the frontend code introduces an additional vulnerability. Identify the class of the vulnerability, give a brief description of it, and write an exploit that results in executing the following JavaScript code as soon as it visit a certain URL: `alert(document.cookie)`.

Please detail all the steps and assumptions you need for a successful exploitation, and write the **complete URL** that triggers the exploit when visited by the victim.

Exercise 3 (8 points)

BeLessSecure, a small IT consultancy company, is rehauling its own local area network. You are in charge of designing the network, taking into account the security requirements.

The company has an external-facing web site, locally hosted on a **web server**, which must be accessible from the public Internet over the standard HTTP and HTTPS ports (80 and 443, respectively). The web site is, actually, a moderately complex web application, which is supported by an **application server** that, in turn, uses a MySQL **database server** as a backend storage. The communication between the web server and the application server happens over TCP, via a standard client/server protocol, where the application server listens on port 8008; the database server listens for connections from the application server on TCP port 3308 (i.e., the standard MySQL port).

Besides the servers, the company network contains the employees' **desktop computers**. Employees can access the application server directly (using the same protocol that the web server uses) through a Windows software pre-installed on each computer by corporate IT. However, employees cannot browse the public web directly: Instead, they must use a **web HTTP(S) proxy** that monitors and filters the web navigation according to corporate policies.

1 [2 point]. Please draw a possible layout for the network described above. Be as precise as possible, and detail all the machines (clients and servers), network zones, and firewalls.



2 [2 points]. Write the firewall rules, assuming firewalls to be stateful packet filters (i.e. you can consider the response rules implicit)

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
<i>FW1 (example)</i>	<i>10.0.0.1 (example)</i>	<i>ANY</i>	<i>zone 1 -> zone 2</i>	<i>192.168.0.2 (example)</i>	<i>443</i>	<i>DENY</i>	<i>(example: the X server in zone 1 cannot contact the Y server)</i>

3. BeLessSecure is **observing anomalous traffic on its network**: They suspect that a nasty cyberattack is taking place. As an incident responder, your task is to understand what is happening. A preliminary analysis reveals that the **anomalous traffic is taking place entirely within the network segment of the employees' computers**. Hence, you connect to the administrative interface of the managed Ethernet switch that connects together all the employees' computers. First of all, you **observe the live network traffic**. You can find below a small excerpt of the network traffic **passing through a single physical port of the switch only** (port name: FastEthernet/12). You also know that there is **only a single device** (a desktop computer with MAC address dc:a9:04:43:21:22) connected to the port FastEthernet/12.

\$ show live-traffic port=FastEthernet/12

```

source MAC          destination MAC      src IP:port    > dest IP:port
-----
a4:da:ca:66:ea:ae   47:c5:a4:11:94:1d   0.0.0.0:58351 > 192.168.1.1:18233
e5:7d:c6:14:24:cc   47:c5:a4:11:94:1d   0.0.0.0:54181 > 192.168.1.1:46945
99:5b:b3:20:90:34   47:c5:a4:11:94:1d   0.0.0.0:39995 > 192.168.1.1:10025
49:e5:8d:59:f6:ea   47:c5:a4:11:94:1d   0.0.0.0:35653 > 192.168.1.1:37071
e0:62:25:3b:0a:20   47:c5:a4:11:94:1d   0.0.0.0:48334 > 192.168.1.1:51807
f3:b9:32:55:04:be   47:c5:a4:11:94:1d   0.0.0.0:17353 > 192.168.1.1:40661
9e:58:7d:51:8e:c7   47:c5:a4:11:94:1d   0.0.0.0:52059 > 192.168.1.1:12306
c4:ac:1c:40:16:c8   47:c5:a4:11:94:1d   0.0.0.0:63469 > 192.168.1.1:50340
6d:9e:8f:60:7b:e0   47:c5:a4:11:94:1d   0.0.0.0:26917 > 192.168.1.1:50171
6b:78:ef:9f:17:2c   47:c5:a4:11:94:1d   0.0.0.0:45635 > 192.168.1.1:36526
<... output omitted ...>
9a:aa:f3:1b:ad:ab   47:c5:a4:11:94:1d   0.0.0.0:35312 > 192.168.1.1:52734

```

Besides the network log, you also take a look at the **content of the switch's CAM table**, as follows:

```
$ show cam-table
CAM Table (memory usage 256KB / total memory 256KB)
-----
MAC Address          Type      Port(s)
-----
82:30:97:69:57:60    DYNAMIC  FastEthernet/1
47:c5:a4:11:94:1d    DYNAMIC  FastEthernet/2
<... output omitted ...>
dc:a9:04:43:21:22    DYNAMIC  FastEthernet/12
a4:da:ca:66:ea:ae    DYNAMIC  FastEthernet/12
e5:7d:c6:14:24:cc    DYNAMIC  FastEthernet/12
99:5b:b3:20:90:34    DYNAMIC  FastEthernet/12
49:e5:8d:59:f6:ea    DYNAMIC  FastEthernet/12
f3:b9:32:55:04:be    DYNAMIC  FastEthernet/12
9e:58:7d:51:8e:c7    DYNAMIC  FastEthernet/12
c4:ac:1c:40:16:c8    DYNAMIC  FastEthernet/12
<... output omitted ...>
9a:aa:f3:1b:ad:ab    DYNAMIC  FastEthernet/12
dc:a9:04:81:3b:e4    DYNAMIC  FastEthernet/24
```

3.1 [2 points]. Which attack is taking place? Give a brief explanation of how it works in general, focusing on (a) what an attacker needs to do in order to implement it, and (b) its effect.

3.2 [1 point]. What is a possible goal of the attacker, in this specific scenario? Justify your answer.

3.3 [1 point]. Describe if, and how, it is possible to mitigate the attack from the point of view of the manufacturer (and, of course, administrator) of the network equipment (e.g., switches, routers, firewalls).

Exercise 4 (7 points)

1. [3 points] Consider anti-analysis techniques used by malware authors. Complete the table below by providing a concise description of each of them and highlighting their differences.

Metamorphism	
Polymorphism	
Packing	

2. A new ransomware variant just broke out, causing a world-wide infection and a huge amount of damages. First of all, you are able to retrieve and disassemble a sample.

address	binary	disassembled instruction
8049177:	53	push ebx
8049178:	83 ec 08	sub esp,0x8
804917b:	b8 07 00 00 00	mov eax,0x7
8049180:	b9 00 00 00 00	mov ecx,0x0
8049185:	0f a2	cuid ²
8049187:	83 e3 0f	and ebx,0xf
804918a:	b8 00 00 00 00	mov eax,0x0
804918f:	83 fb 01	cmp ebx,0x1
8049192:	74 05	je 8049199
8049194:	e8 cd ff ff ff	call 8049166 <encrypt_disk>
8049199:	83 c4 08	add esp,0x8
804919c:	5b	pop ebx
804919d:	c3	ret

² Excerpt from the Intel IA-32 manual: “CUID returns processor identification and feature information in the EAX, EBX, ECX, and EDX registers. The instruction’s output is dependent on the contents of the EAX register upon execution (in some cases, ECX as well).”

Additionally, you execute the malware sample in a widespread dynamic analysis environment that, while the program is executing, collects the list of system calls invoked. The log of system calls provided by the sandbox is as follows:

```
execve("./ransomware", ["./ransomware"], 0x7ffd96b406b0 /* 33 vars */) = 0
strace: [ Process PID=372 runs in 32 bit mode. ]

[syscalls during program load, set up of stack/heap/... omitted for the sake of brevity]

exit_group(0)                                = ?
+++ exited with 0 +++
```

2.1. [2 points] Looking at the dynamic traces, it seems that the ransomware is inactive (i.e., it simply terminates once executed). What technique is the ransomware implementing? How the technique works?

2.2. [2 points] Consider now malware analysis techniques. Briefly discuss which is the best technique to detect the malware behavior implemented by the ransomware considered before.